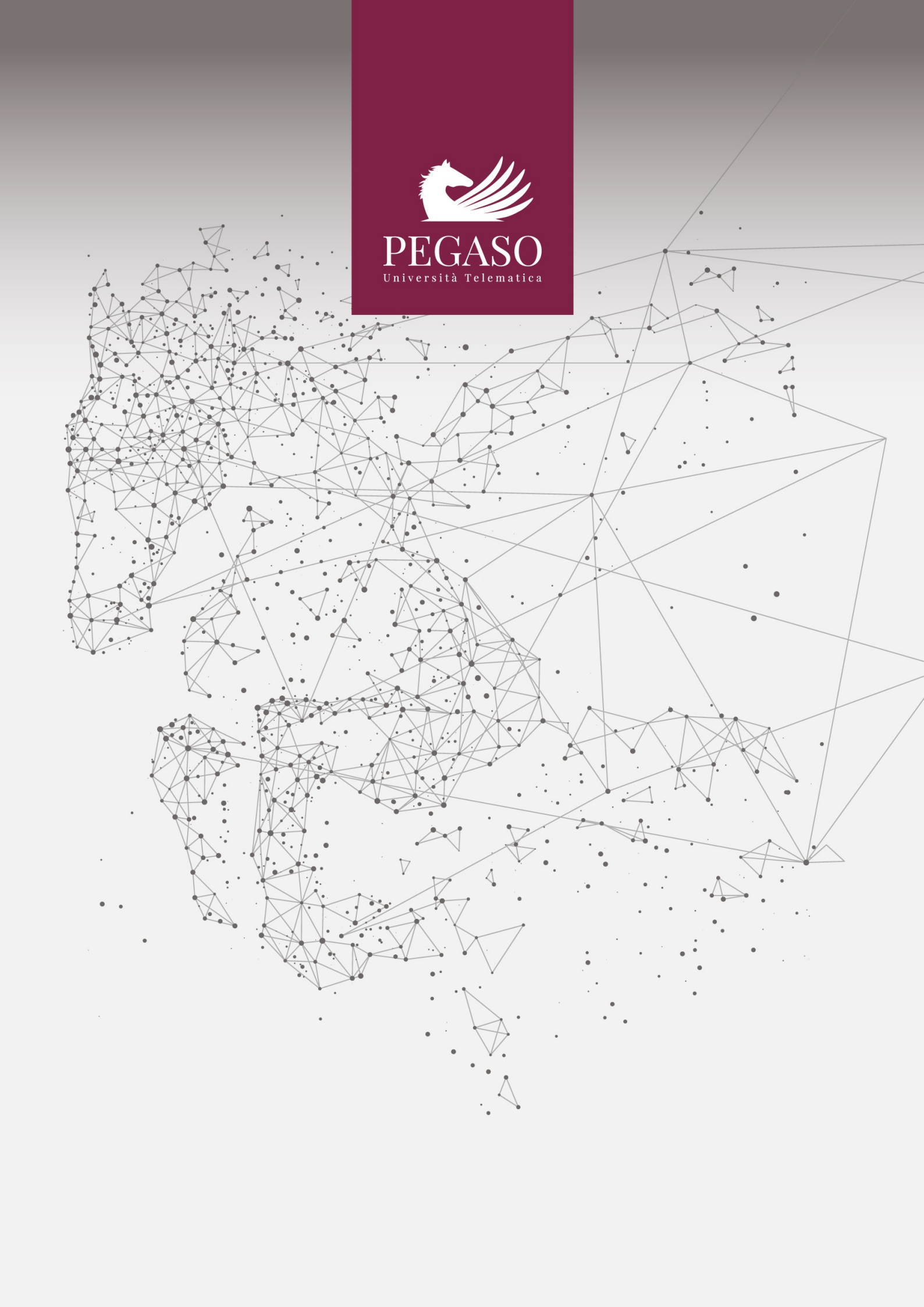

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. ARCHITETTURE A STRATI....5
3. PROTOCOLLI DI COMUNICAZIONE STRATIFICATA....7
4. STRATIFICAZIONE DELLE APPLICAZIONI....9
BIBLIOGRAFIA....11

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

Inizieremo la discussione sulle architetture esaminando prima l'organizzazione logica di un sistema distribuito in componenti software, definita anche architettura software. A causa dei notevoli progressi compiuti nello studio delle architetture software, è ormai generalmente risaputo che lo sviluppo di grandi sistemi software richiede la progettazione e l'impiego di un'architettura perché esso possa funzionare con successo.

Nel contesto di questa lezione, il concetto di stile architettonico è essenziale. Un concetto di questo tipo è descritto in termini di componenti, il modo in cui tali componenti sono collegati tra loro, le informazioni che vengono passate tra i componenti e, infine, il modo in cui questi elementi sono configurati collettivamente in un sistema. Un componente è un'unità modulare sostituibile che ha interfacce ben definite e può funzionare in modo indipendente all'interno del suo ambiente. È essenziale che un componente possa essere sostituito, in particolare mentre un sistema è ancora in funzione, al fine di garantire l'affidabilità. Ciò è dovuto al fatto che in molti casi, lo spegnimento di un sistema allo scopo di eseguire la manutenzione non è un'opzione percorribile. Nel peggiore dei casi, solo alcuni suoi componenti potrebbero essere resi temporaneamente inutilizzabili. Per sostituire correttamente un componente, le sue interfacce devono poter rimanere invariate in ogni momento.

I connettori sono generalmente descritti come meccanismi che mediano la comunicazione, il coordinamento o la cooperazione tra i componenti. Ciò rende la comprensione del concetto di connettore un po' più impegnativa rispetto a quella del semplice componente. Ad esempio, un connettore potrebbe essere creato per le RPC, lo scambio dei messaggi o lo streaming di dati. I connettori, in altre parole, consentono il flusso di messaggi di controllo e dati attraverso i diversi componenti.

Mediante l'uso di diversi componenti e connettori, siamo in grado di ottenere un'ampia varietà di configurazioni, che possono quindi essere classificate come diversi stili architettonici. Ci sono stati numerosi stili identificati fino ad oggi, i più essenziali per i sistemi distribuiti sono:

- Architetture stratificate
- Architetture basate su oggetti
- Architetture basate su risorse
- Architetture basate su eventi

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La maggior parte dei sistemi distribuiti del mondo reale combina un'ampia varietà di stili architettonici. Poiché seguire un approccio che divide un sistema in diversi livelli (logici) è un principio universale, si può osservare questa tendenza nella maggior parte degli altri stili architettonici.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Architetture a Strati

In una architettura a strati, i componenti sono organizzati in modo stratificato, con un componente ad un livello superiore (layer L_i) che è in grado di effettuare una downcall ad un componente ad un livello inferiore (layer L_i), e generalmente si aspetta una risposta da quest'ultimo. Una chiamata a una componente di livello superiore sarà effettuata solo in circostanze straordinarie.

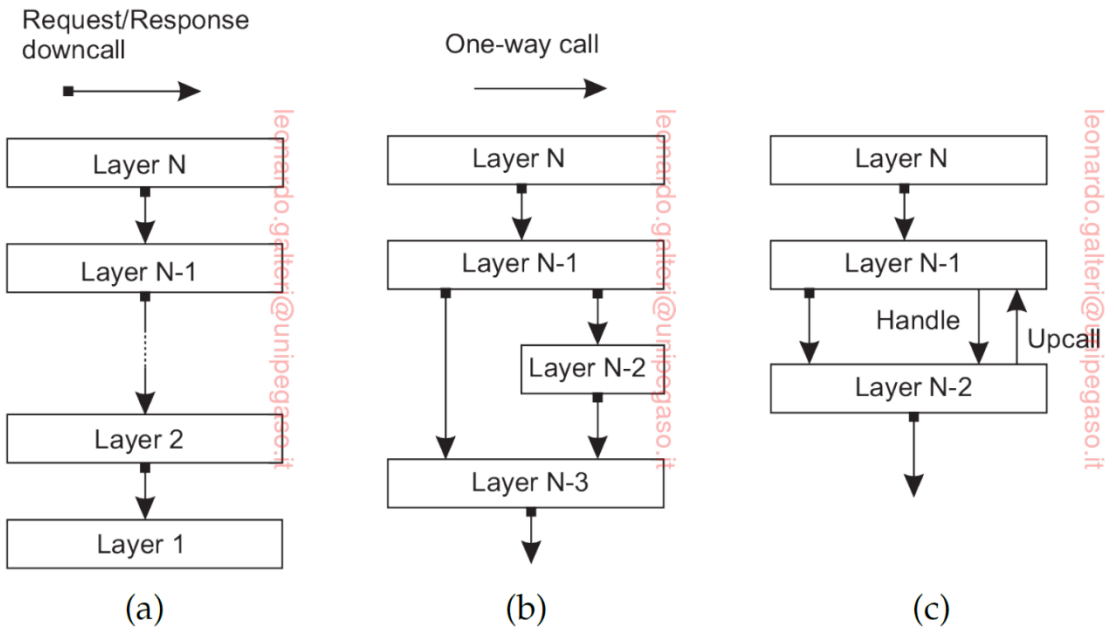


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Nella Figura 1 vengono illustrati i tre scenari più comuni che possono verificarsi. Una tipica struttura organizzativa è illustrata nella Figura 1(a), che mostra che le chiamate verso strati inferiori sono le uniche chiamate che possono essere effettuate in questa struttura. Quando si tratta di contesti in cui avvengono comunicazioni di rete, questa struttura viene spesso impiegata con successo.

In molti contesti diversi, ci imbattiamo anche nella struttura organizzativa raffigurata nella Figura 1(b). Prendiamo ad esempio un'applicazione denominata A che, per interfacciarsi con un sistema operativo, utilizza una libreria chiamata L_{OS}. L'applicazione fa uso di una libreria matematica specializzata chiamata L_{math}, che è stata implementata facendo uso anche di L_{OS}. In questo caso, sempre riferendosi alla Figura 1(b), A è implementata sul livello N-1, L_{math} è implementata sul livello N-2 e L_{OS}, che è condivisa da entrambi, è implementata sul livello N-3.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, un caso speciale è presentato nella Figura 1(c). Può essere più efficiente che uno strato inferiore esegua una chiamata al suo livello superiore successivo in determinate circostanze. Un esempio ciò potrebbe essere il caso in cui un sistema operativo rileva il verificarsi di un evento e, in risposta, chiama un'operazione definita dall'utente per la quale un'applicazione ha già fornito un riferimento. Si tratta di uno scenario anomalo, in genere denominato *handle*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Protocolli di comunicazione stratificata

L'architettura stratificata dei cosiddetti stack di protocolli di comunicazione è ben nota e ampiamente utilizzata in tutto il mondo.

Ogni livello di uno stack di protocolli di comunicazione è responsabile dell'implementazione di uno o più servizi di comunicazione che consentono la trasmissione di dati da una sorgente a una o più destinazioni. Per raggiungere questo obiettivo, ogni livello fornisce un'interfaccia che delinea le diverse funzioni che possono essere chiamate. In generale, l'interfaccia di un servizio dovrebbe nascondere l'effettiva implementazione del servizio nella sua interezza. Quando si parla di comunicazione, un'altra idea essenziale da tenere a mente è quella di un protocollo di comunicazione, che definisce le linee guida che ogni parte deve rispettare prima di poter condividere informazioni tra loro. È necessario avere una solida comprensione delle distinzioni tra un servizio fornito da un livello, l'interfaccia attraverso la quale viene concesso l'accesso a tale servizio e il protocollo che un livello utilizza per stabilire la comunicazione con altri livelli. La figura 2 illustra questa distinzione.

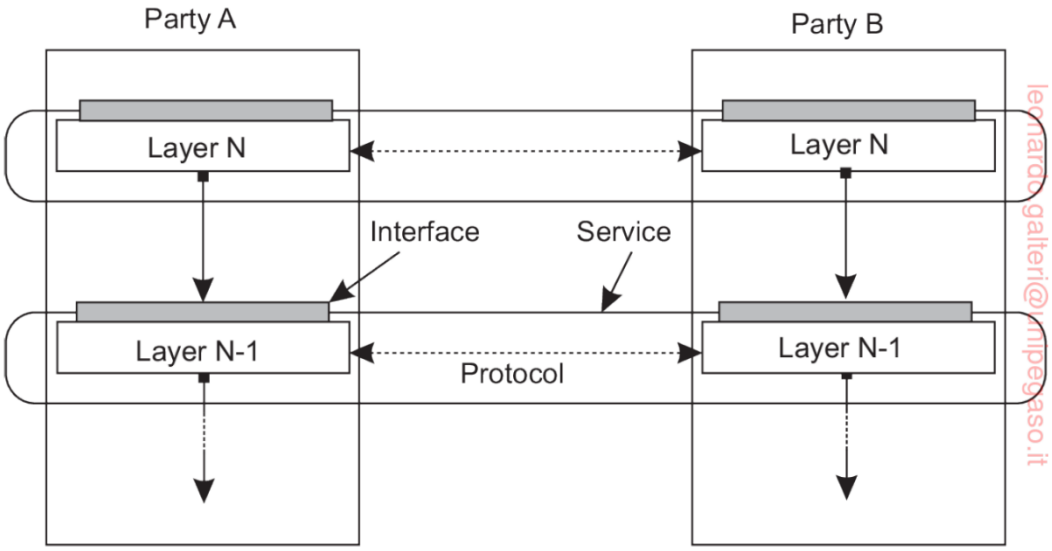


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Si consideri un servizio affidabile e orientato alla connessione, che è qualcosa che molti diversi tipi di sistemi di comunicazione offrono, in modo da poter comprendere nel dettaglio la differenza tra le entità. In questo scenario, la capacità di una parte di comunicare con un'altra deve essere preceduta dalla creazione di una connessione tra le due parti prima che una delle parti possa inviare o ricevere messaggi.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Essere affidabili significa che verranno fornite garanzie che i messaggi inviati saranno effettivamente consegnati all'altra parte, anche quando c'è un alto rischio che tali messaggi possano andare persi. Inoltre, tali servizi in genere garantiscono inoltre che i messaggi vengano recapitati nello stesso ordine in cui sono stati inviati.

Attraverso l'utilizzo del Transmission Control Protocol (TCP), questo tipo di servizio è reso possibile attraverso Internet. Il protocollo specifica quali messaggi devono essere scambiati quando si stabilisce o si interrompe una connessione, nonché cosa è necessario fare per preservare l'ordine dei dati trasferiti e cosa devono fare entrambe le parti per rilevare e correggere i dati persi durante la trasmissione. Inoltre, il protocollo delinea come rilevare e correggere i dati persi durante la trasmissione. Questo servizio è reso disponibile agli utenti sotto forma di un'interfaccia di programmazione relativamente semplice, che include chiamate per stabilire una connessione, inviare e ricevere messaggi e terminare la connessione una volta che ha raggiunto il suo scopo. In pratica, sono disponibili numerose interfacce, la cui selezione è spesso determinata dal sistema operativo o dal linguaggio di programmazione utilizzato. Allo stesso modo, ci sono numerosi modi differenti in cui il protocollo e le sue interfacce possono essere implementati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Stratificazione delle applicazioni

La stratificazione logica delle applicazioni è dove concentreremo l'attenzione a questo punto. Molte persone hanno sostenuto la distinzione tra tre livelli logici, essenzialmente aderendo a uno stile architettonico stratificato, alla luce del fatto che una grande classe di applicazioni distribuite ha lo scopo di supportare l'accesso degli utenti o delle applicazioni ai database. I tre livelli logici sono classificati in:

- Livello dell'interfaccia dell'applicazione
- Livello di elaborazione
- Livello dei dati

Assumendo questa classificazione per quanto riguarda la stratificazione, possiamo vedere che le applicazioni possono spesso essere costruite da circa tre parti distinte: una parte che gestisce l'interazione con un utente o qualche altra applicazione esterna, una parte che opera su un database o filesystem e una parte centrale che generalmente contiene le funzionalità principali dell'applicazione. La collocazione del livello di elaborazione più sensata è quella in cui essa si trovi nel mezzo agli altri due. A differenza delle interfacce utente e dei database, il livello di elaborazione non ha molti aspetti in comune con altri livelli. Di conseguenza, al fine di rendere questo livello più comprensibile, si ricorre all'illustrazione di alcuni esempi.

Si pensi a un motore di ricerca per Internet come primo esempio. Se si ignorano tutti i banner animati, le immagini e altre finestre decorative, l'interfaccia utente di un motore di ricerca può essere molto semplice: un utente digita una stringa di parole chiave e quindi gli viene fornito un elenco dei titoli di diverse pagine web. Il backend è composto da un enorme database di pagine Web che sono state precaricate e indicizzate prima di essere visualizzate. Il cuore del motore di ricerca è un software che, data una stringa di parole chiave da parte dell'utente, genera una o più query per il database corrispondente. Successivamente, ordina i risultati in un elenco e quindi converte quell'elenco in un gruppo di pagine HTML. Questa parte del processo, che recupera le informazioni, si trova in genere a livello di elaborazione.

Si consideri, come secondo esempio, un sistema di supporto decisionale progettato specificamente per l'intermediazione azionaria. Un sistema come questo può essere suddiviso in tre livelli distinti, proprio come un motore di ricerca, e sono i seguenti:

- Un front-end che implementa l'interfaccia utente o fornisce un'interfaccia di programmazione dell'applicazione a programmi di terze parti.
- Un back-end che consente agli utenti di accedere a un database che contiene i dati finanziari.
- I programmi di analisi che risiedono tra questi due.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

È possibile che l'analisi dei dati finanziari richieda alcune tecniche e metodi avanzati nei campi della statistica e dell'intelligenza artificiale. Verosimilmente il nucleo di un sistema di supporto alle decisioni finanziarie deve essere eseguito su computer ad alte prestazioni in determinate circostanze. Ciò è necessario per ottenere il throughput e la reattività che ci si aspetta dagli utenti del sistema.

Come esempio finale, si prenda in considerazione un tipico pacchetto desktop, che include un elaboratore di testi, un'applicazione per la creazione di fogli di calcolo, servizi di comunicazione e così via. Queste suite "office" in genere integrano i loro vari componenti attraverso un'interfaccia utente unificata che consente la gestione integrata dei documenti ed esegue operazioni sui file recuperati dalla home directory dell'utente. Nel contesto di questo esempio, il livello di elaborazione è costituito da una raccolta piuttosto ampia di programmi, ognuno dei quali possiede capacità di elaborazione relativamente basilari.

I programmi responsabili della manutenzione dei dati effettivi su cui operano le applicazioni possono essere trovati a livello di dati. Il fatto che i dati siano spesso persistenti è una caratteristica importante di questo livello. Ciò significa che anche se l'utente non ha alcuna applicazione in esecuzione, i dati verranno comunque archiviati da qualche parte per un uso successivo. È pratica comune utilizzare un database a tutti gli effetti, nonostante il fatto che il livello dei dati possa, nella sua manifestazione più elementare, essere rappresentato da un file system.

Oltre a memorizzare semplicemente i dati, il livello di dati è in genere anche responsabile di garantire che i dati siano coerenti tra le varie applicazioni. Quando vengono utilizzati i database, il mantenimento della coerenza richiede l'archiviazione dei metadati a questo livello. Questi metadati possono includere descrizioni di tabelle, vincoli di immissione e metadati specifici dell'applicazione utilizzata. Quando il debito della carta di credito di un cliente raggiunge un certo valore, ad esempio, la banca potrebbe voler inviare una notifica al cliente. Questo tipo di informazioni può essere mantenuto aggiornato utilizzando un trigger di database che, al momento opportuno, metterà in azione uno specifico gestore corrispondente a tale trigger.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Bass L., Clements P., and Kazman R. Software Architecture in Practice. Addison-Wesley, Reading, MA., 2nd edition, 2003.
- Mehta N., Medvidovic N., and Phadke S. Towards A Taxonomy Of Software Connectors. In 22nd International Conference on Software Engineering, 2000
- Shaw M. and Clements P. A Field Guide to Boxology: Preliminary Classification of Architectural Styles for Software Systems. In 21st International Computer Software and Applications Conference, 1997.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).